

Wiring Beats Blending: What Transfers Between Transformer Sizes — and What Doesn't

Anonymous authors
Paper under double-blind review

Abstract

Model families train every size from scratch. Can a pretrained large model be converted into a smaller sibling? We characterize the 1.4B→410M conversion in the Pythia family end-to-end: (i) representations align strongly across sizes (ridge $R^2 = 0.84$) while parameters align weakly; (ii) dense weight projection is functionally destructive — provably not an assembly artifact — because basis mixing breaks rotary, per-head, GELU, and Layer-Norm structure; (iii) after the best-fit linear operator, weight residuals are statistically indistinguishable from noise under shuffle controls; (iv) conversion value therefore lives in initialization: in matched-budget continued pre-training we decompose conversion into two independent levers — least-squares compensation (function: best zero-shot) and variance-preserving rescale (training dynamics: best endpoints) — whose combination dominates the strongest subcloning variant on both axes and in every seed (83.9 ± 1.8 vs. 89.7 ± 3.7 final perplexity, 3/3 paired wins; $18\times$ better than from-scratch at 30M tokens). The ordering holds out-of-domain and at $2\times$ the training context, and the $\sim 13\times$ margin over from-scratch generalizes with three seeds to a held-out, depth-dominated pair — where selection variants tie, mapping the boundary of compensation to the width-reduced regime. Code, checkpoints, and the frozen evaluation corpus are released.

1 Introduction

Transformer families ship as ladders of discrete sizes — Pythia alone spans 70M to 12B parameters (Biderman et al., 2023) — because different deployment budgets need different models. Under current practice every rung is bought separately: each size is a full pretraining run with its own GPU and energy bill, even though the family members are trained on identical data with a shared tokenizer and architecture, differing only in width and depth. The siblings are, by construction, solutions to the same problem at different capacities; training each from nothing treats them as unrelated. This paper asks the question the ladder’s cost makes practical and its construction makes tractable: what actually transfers between two sizes of the same pretrained model family — and through what operation can it be carried into the smaller shape?

We answer end-to-end on the Pythia suite, where identical data and tokenizer isolate size as the only variable (§3), in four controlled steps. First, we locate the cross-size relation by measuring alignment separately in representations and in parameters (§4): the smaller model’s activations are largely a linear image of the larger model’s (held-out ridge $R^2 = 0.84$), while its weights are not ($R^2 = 0.25\text{--}0.39$ even for embeddings, the cleanest case). Second, we diagnose why the natural exploit of that alignment — densely projecting the large weights into the small shape — is functionally destructive (§5): a bit-exact reconstruction control rules out assembly, leaving the projection mathematics itself, whose basis mixing breaks rotary, per-head, GELU, and LayerNorm structure — blending destroys the wiring that structured selection preserves. Third, we test whether what the best-fit linear operator misses is a structured, learnable correction (§6): under shuffle controls it is statistically indistinguishable from noise, so no zero-shot fix exists at this granularity. Conversion value must therefore live in initialization, and the fourth step is a matched-budget recovery race (§7): every candidate init is built without access to the target’s weights and trained under

identical data order, schedule, and token budget. The race decomposes conversion into two independent levers over structure-respecting selection — closed-form least-squares compensation, which repairs what the initial network computes, and variance-preserving rescale, which repairs the scale the optimizer sees — and shows that, because they act on disjoint weight paths, they stack.

Our contributions are:

- A controlled characterization of what transfers across sizes. Within a family trained on identical data, representations align strongly under a linear map (held-out ridge $R^2 = 0.84$) while parameters align weakly ($R^2 = 0.25$ – 0.39 in the cleanest, embedding case) — the gap the rest of the paper pivots on (§4).
- A mechanically verified diagnosis of why dense projection fails. Rebuilding the donor from its own extracted weights reproduces its logits bit-exactly, so the projected model’s collapse is attributable to basis mixing alone: dense maps break rotary pairing, per-head attention, elementwise GELU, and per-feature LayerNorm — with the LayerNorm mismatch isolated as the largest single factor (§5).
- A null result under controls. After the best-fit linear operator, weight residuals carry no cross-layer direction, no patch-learnable signal, and no cross-pair transfer (spectral, consistency, learnability, and transfer tests against shuffle controls with bootstrap confidence intervals, $p \geq 0.99$): a learned zero-shot correction is not available at this granularity (§6).
- A two-lever decomposition of conversion, and a method that stacks them. At matched budget, least-squares compensation is the function lever (best zero-shot) and variance-preserving rescale is the training-dynamics lever (best endpoints); their combination — Compensated Selection — dominates the strongest variant of weight subcloning (Samragh et al., 2023) on both axes and in every seed (83.9 ± 1.8 vs. 89.7 ± 3.7 final perplexity, 3/3 paired wins; zero-shot 18.5k vs. 61.9k) and finishes $18\times$ better than from scratch at 30M tokens; the ordering holds out-of-domain and at $2\times$ the training context, and the margin over from scratch ($\sim 13\times$) generalizes with three seeds to a held-out depth-dominated pair (§7).

The paper reads as the argument above: §4 establishes the representation–parameter gap, §5 diagnoses dense projection, §6 closes the door on zero-shot residual correction, and §7 runs the conversion race and its hardening, with related work in §2, the experimental setup in §3, and limitations in §8.

2 Related work

Downscaling pretrained models. The closest prior work is Weight Subcloning (Samragh et al., 2023), which initializes a smaller transformer by ranking units of a larger pretrained one, copying the selected rows and columns, and rescaling — establishing that selection-based initialization accelerates training of the target size. Sheared LLaMA (Xia et al., 2024) learns structured pruning masks jointly with continued pre-training and produces strong small models at a fraction of from-scratch compute; it optimizes the full pipeline, whereas we hold the training recipe fixed across arms and isolate what the initialization alone contributes. Our closed-form compensation belongs to the pruning-with-reconstruction lineage: adjusting surviving weights to absorb the function of removed ones dates to Optimal Brain Surgeon (Hassibi & Stork, 1993) and reappears in structured LLM pruning such as LLM-Pruner (Ma et al., 2023); we apply the reconstruction step once, in closed form, at conversion time. Knowledge distillation (Hinton et al., 2015) transfers function through a teacher’s outputs and requires a full training run; we study the complementary question of what transfers through weights at negligible cost, and the two are combinable.

Growing pretrained models. The mirror direction — initializing a larger model from a smaller one — has a longer history: Net2Net (Chen et al., 2016) introduced function-preserving widening and deepening, bert2BERT (Chen et al., 2022) adapted it to transformers, staged training (Shen et al., 2022) formalized growth operators that preserve loss

and training dynamics, and LiGO (Wang et al., 2023) learns a structured linear map from the small model’s weights to the large initialization. That linear operators carry useful signal upward is consistent with our finding that, downward, even a fitted dense projection initializes far better than random; the asymmetry we document is that dense basis mixing destroys the rotary, per-head, and normalization structure that selection preserves (§5).

Representation similarity and parameter symmetries. Our analyses use standard similarity tools: CKA (Kornblith et al., 2019) and SVCCA (Raghu et al., 2017) measure alignment between representations, and model stitching (Lenc & Vedaldi, 2015; Bansal et al., 2021) tests functional interchangeability through a trained adapter. Work on permutation symmetries and model merging (Ainsworth et al., 2023) shows that networks are naturally compared modulo the transformations under which the architecture is invariant. Our selection-versus-blending result is an instance of the same principle: structured selection composes a permutation — an element of the architecture’s symmetry group, restricted to respect head boundaries and rotary frequency pairs (Su et al., 2024) — with coordinate deletion, so every surviving unit keeps the exact nonlinear and positional semantics the architecture assigns it; dense projection mixes coordinates, exits the group, and is punished for it.

Positioning. Each ingredient above exists in isolation: selection-based initialization, least-squares reconstruction, linear growth operators, and representation similarity analysis. What is new here is the setting — conversion between sizes of a single model family trained on identical data, removing data and tokenizer confounds — together with rotary-frequency-matched selection and a controlled end-to-end characterization, from representation alignment through projection diagnosis and residual null tests to a matched-budget training race. Because our subcloning baseline re-implements the recipe of Samragh et al. (2023), we disclose three fidelity differences and label the baseline subcloning-style: (a) we score attention heads and MLP units by weight norms and residual lanes by activation variance, where they score by activation magnitudes; (b) when depth must shrink we remove blocks at even stride rather than from the middle; and (c) we evaluate their $\sqrt{d/d'}$ weight rescale directly rather than adopting it wholesale (§7) — applied to every matrix it collapses zero-shot quality because LayerNorm already renormalizes most read paths, while its variance-preserving motivation holds exactly on the projections no norm protects, the same two families our compensation re-fits.

3 Experimental setup

Model family and conversion pairs. We study the Pythia suite (Biderman et al., 2023): a family of GPT-NeoX models trained on identical data (the Pile) with a shared tokenizer, which isolates size as the only variable between family members. Our primary pair converts 1.4B $\rightarrow$ 410M (24 layers and 16 heads in both; only widths shrink: residual 2048 $\rightarrow$ 1024, head dim 128 $\rightarrow$ 64, MLP 8192 $\rightarrow$ 4096), so layer correspondence is trivially one-to-one and the analysis isolates width. A held-out pair, 410M $\rightarrow$ 160M, exercises the axes the primary pair does not: depth halves (24 $\rightarrow$ 12 blocks), the head count drops (16 $\rightarrow$ 12; head dim unchanged), and width cuts are mild ($\sim$ 25%). A third pair, 6.9B $\rightarrow$ 1.4B, tests scale (all three axes reduced; §7).

Frozen evaluation corpus. All representation analyses probe every model with the same inputs: 10,000 sequences of 128 tokens drawn from the Pile (pile-uncopyrighted), tokenized once and frozen for the entire project. The first 500 sequences (all token positions) form the subset for similarity analysis (§4); the remainder contribute mean-pooled and last-token activations. Continued pre-training streams the same corpus but skips the region used to build the frozen set (a contamination guard), and each random seed trains on a disjoint stream offset.

Behavioral evaluation. Language-modeling quality is strided sliding-window perplexity on WikiText-103 validation (every token scored exactly once). Hardening evaluations add C4 perplexity (out-of-domain), perplexity at $2\times$ the training context (2048), and zero-shot

accuracy on LAMBADA, ARC-Easy, HellaSwag, and PIQA via lm-eval. Reference models anchor every table; anchor scores match published Pythia numbers.

Determinism and verification. All forward passes are greedy and seeded; repeated runs are bit-identical on the same device. Weight extraction is validated by exact reconstruction: the fused GPT-NeoX QKV matrices are split head-aware and re-fused to bit-identical equality, and a model rebuilt from its own extracted weights reproduces the original logits with zero difference (§5) — any quality change downstream is therefore attributable to conversion mathematics, not assembly.

Training protocol. Every conversion arm within an experiment shares identical data order, optimizer (AdamW, $\beta = (0.9, 0.95)$, weight decay 0.1), cosine schedule with 100-step warmup, gradient clipping at 1.0, bf16 autocast, and token budget (30M primary; 100M persistence checks); the learning rate equals the target size’s original pretraining rate. Initialization construction costs are negligible against any training budget (selection: seconds; moments pass for compensation: ~ 75 s; closed-form solves: seconds) and are included in the compute accounting. All experiments ran on a single NVIDIA GB10 (128 GB unified memory); any 24 GB+ CUDA GPU reproduces them (Appendix TODO).

4 Representations align, parameters do not

A size conversion presupposes that the two family members are related; the question is where the relation lives. On the primary width-only pair (1.4B $\rightarrow$ 410M, same depth and head count; §3) we measure the relation in two places — the representations both models compute, and the parameters they store — and find it in only one of them.

Layer correspondence saturates under CKA. Linear CKA between all 24×24 layer pairs, computed on the similarity subset (all token positions; §3), gives a diagonal mean of 0.883, yet only 12% of layers place their single best match exactly on the diagonal. The reason is visible in Figure 1 (left): the heatmap is one large saturated block — layers 0–2 distinct, the middle band (layers ~ 4 –22) mutually ≈ 1.0 , layer 23 distinct. Linear CKA therefore confirms only coarse early/mid/late correspondence and saturates in the middle band (a known limitation of the metric), so it is not a sharp per-layer discriminator. This motivates a directional, quantitative map: how much of the small model’s representation is a linear image of the large model’s?

Activations are largely a linear image across widths. Per layer, we fit a large $\rightarrow$ small map on pooled activations (8k train / 2k held-out) and score held-out R^2 under two operators of increasing freedom: Procrustes (a rotation plus one global scale) and ridge (a full linear map $2048 \rightarrow 1024$). Ridge reaches a mean held-out $R^2 = 0.844$ (Table 1): the 410M pooled representation is largely a linear projection of the 1.4B’s, capturing 84% of held-out variance. Procrustes already reaches 0.717, so most of the relationship is a change of basis; the ~ 0.13 gap is genuine non-orthogonal reshaping. Both maps are discriminative per layer and dip together at layers 3–5 — the same layers CKA flagged as distinct (Figure 1, right).

Raw parameters do not. The token embeddings are the one weight matrix where the two sizes share an axis (the vocabulary) and differ only in width, so a single map applies; we split rows (tokens) 80/20. Even in this cleanest case a linear map explains only 0.25–0.39 of weight variance (Table 1): ridge $R^2 = 0.248$ on the input embedding and 0.386 on the output embedding, against 0.844 for activations.

The representation–parameter gap. Two independently trained members of a family arrive at closely related representations while their parameters are only loosely a linear map of one another. This gap is the pivot of the paper. If representations align but weights do not, then projecting the large model’s weights into the small model’s shape (§5) must leave a substantial residual, and whether that residual is structured and learnable or merely noise (§6) decides where conversion value can come from.

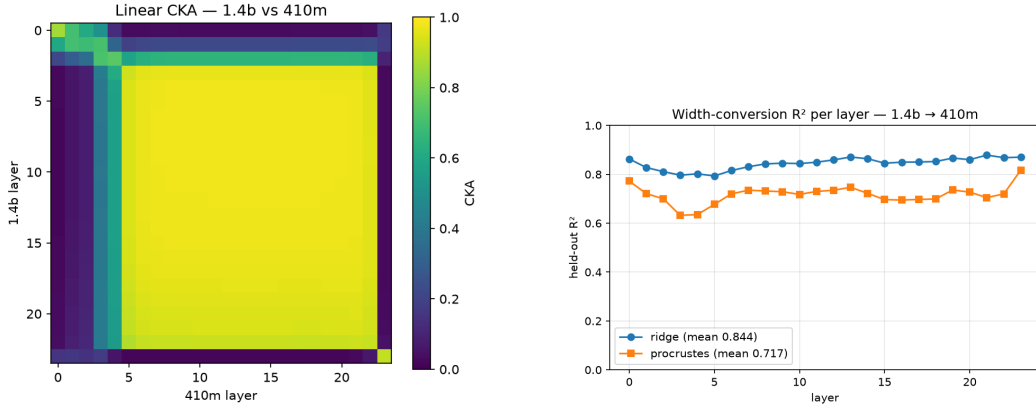


Figure 1: Alignment across sizes for the primary pair (1.4B $\rightarrow$ 410M). (left) Linear CKA between all 24×24 layer pairs: a saturated middle band (layers ~ 4 –22 mutually ≈ 1.0) confirms coarse correspondence but resolves no sharp per-layer match. (right) Held-out R^2 of the per-layer activation maps: ridge (full linear) and Procrustes (rotation+scale) track each other and dip together at layers 3–5, the layers CKA also flags as distinct.

Table 1: Alignment across sizes, held-out. Activations align strongly under a linear map; raw parameters, even in the cleanest (embedding) case, do not. Best (highest) map per row in bold.

Target	CKA	Ridge R^2	Procrustes R^2
Pooled activations (per-layer mean)	0.883 [†]	0.844	0.717
Input embedding	0.512	0.248	0.167
Output embedding (unembedding)	0.678	0.386	0.242

[†] For activations, the diagonal mean of the 24×24 linear-CKA matrix, which saturates in the middle band (Figure 1, left); for embeddings, linear CKA before mapping.

5 Why dense projection fails

If representations align across sizes, the natural attempt is to project the large model’s weights into the small model’s shape and read off a working small model. We formulate this projection using only the size relationship (never the target’s weights), measure the residual, run the assembled model, and diagnose why it breaks. The verdict is that dense blending destroys the network’s structured computation — and, via a bit-exact control, that the destruction is in the projection mathematics, not the assembly.

Target-free projection. Each weight is mapped by a pair of structured linear operators,

$$\hat{W} = P_{\text{out}} W P_{\text{in}}^{\top}, \quad P_{\bullet} \in \{P_{\text{res}}, P_{QK}, P_{VO}, P_{\text{MLP}}\}, \quad (1)$$

whose bases are built only from the large model and the size relationship, never from the 410M target. The residual-stream axis uses a global activation-PCA basis P_{res} ; the internal axes use SVD bases shared between interacting weights, so that composed operations stay consistent — Q and K share one basis, V and O share one, and MLP_UP/MLP_DOWN share a single joint basis computed from the column stack $[\text{UP}, \text{DOWN}^{\top}]$. Sharing is not cosmetic: a per-weight basis for the MLP hidden axis cannot span the target, since one 8192×2048 matrix has rank $\leq 2048 < 4096$, so the interacting pair must be factorized together.

The projected weights miss the target solution. Table 2 reports the relative Frobenius error of $\hat{W}$ against the actual 410M weights, averaged over 24 layers, for the target-free

Table 2: Relative Frobenius error of projected weights vs. the actual 410M weights, mean over 24 layers. Every target-free error exceeds 1 (worse than predicting zero). Lower error per row in bold.

Weight	Projection (target-free)	Operator (fitted) [†]
Q	1.471	0.658
K	1.591	0.758
V	1.390	0.804
O	1.724	0.795
MLP_UP	1.668	0.683
MLP_DOWN	1.723	0.686

[†] A single (A, B) shared across layers per type; it peeks at the target and so upper-bounds the linearly explainable fraction. A per-layer fit is degenerate (error $\rightarrow 0$ around any full-rank W) and is not reported.

projection of Eq. 1 and — as a diagnostic upper bound — a shared operator (A, B) fitted with access to the target. Every target-free error exceeds 1, i.e. worse than predicting zero: the projected weights share almost nothing with the 410M’s particular solution, confirming the activation-versus-weight gap of §4 at every layer. Even the fitted operator, which peeks at the target, only reaches 0.66–0.80; at best $\sim 30\%$ of weight variance is linearly explainable across sizes with one shared map.

Behavioral anchors. We assemble a 410M-shaped model from the projected 1.4B weights and score strided WikiText-103 perplexity against reference and anchor models (Table 3). Greedy generation is deterministic but word salad, and the acceptance bar — beat random init — is not met zero-shot. The shape of the failure is instructive: random init produces near-uniform logits (perplexity on the order of the vocabulary size), whereas the projected model is confidently wrong, which scores far worse than uniform.

The failure is projection math, not assembly. Before diagnosing the mechanism, we rule out a plumbing bug. Rebuilding the 1.4B from its own extracted weights with no projection reproduces the real model’s logits with $\max |\text{diff}| = 0$ — bit-exact (previewed in §3). Extraction, head-aware QKV split/fuse, embeddings, and LayerNorm plumbing are all correct, so the entire quality loss in Table 3 is attributable to Eq. 1.

Structure mixing breaks the computation. A dense basis change on any internal axis silently changes what the block computes, because every structured or nonlinear operation is tied to specific coordinates. Rotary embeddings act on fixed per-head dimension pairs, so any head-space mixing distorts positions. Attention is per-head, yet our global $2048 \rightarrow 1024$ head-space basis mixes dimensions across heads. GELU is elementwise in the MLP hidden space, so $\text{GELU}(Bx) \neq B \text{GELU}(x)$. LayerNorm gains and biases are per-feature in the residual stream, so a rotated stream no longer matches any diagonal gain — which is why the projected-LN variant explodes to 10^{13} . The consequence is that function-preserving zero-shot conversion needs structure-respecting selection maps — keep or drop whole heads, hidden units, and features, the architecture’s own symmetry group — rather than dense blending. Dense learned projections are thus useful as initializations for brief fine-tuning, not as drop-in models; §7 takes up exactly that comparison.

6 Post-operator residuals are noise

Section 5 shows that even the strongest shared linear operator explains at best $\sim 30\%$ of weight variance (relative error 0.66–0.80), leaving the majority as a residual. Is that residual a structured, learnable correction — or noise? We test it against matched controls, under the strongest-alignment condition so the verdict is conservative: if any signal survives the best linear map, these tests should find it.

Table 3: Zero-shot language-modeling quality of the dense-projected model (strided WikiText-103 perplexity). The projected model does not beat random init. Best (lowest) in bold.

Model	Perplexity
real 1.4B (reference)	11.5
real 410M (upper anchor)	15.6
random-init 410M (lower anchor)	65,962
projected, LN copied from 410M	181,242
projected, LN projected from 1.4B	10^{13}

With fresh neutral LayerNorms (gain 1, bias 0) the same projected matrices score $\sim 12.3k$ quick-perplexity — $\sim 15\times$ better than the 181,242 row and $\sim 5\times$ better than random init — so much of the collapse is LayerNorm mismatch, not weight destruction alone (§7); the structure-mixing diagnosis still holds directionally.

The residual under the strongest operator. We define the residual per weight as

$$\Delta(l, \text{type}) = W_{410M} - A W_{1.4B} B^\top, \quad (2)$$

using the shared fitted operator (A, B) per type — the strongest-alignment condition of §5, so whatever remains is what no single linear size-map can explain. Four tests probe Δ , each against a matched control.

Test 1 — spectral (effective rank). Table 4 compares the effective rank of Δ against a per-type shuffled control and a shape/scale-matched Gaussian control. Δ sits 3–6% below both controls in every type: a faint spectral concentration, so the residual is not perfectly isotropic noise (Figure 2).

Test 2 — cross-layer consistency. The mean pairwise cosine of vectorized Δ across layers is $\approx \pm 0.0002$ in every type, identical to the shuffle control: no direction is shared across layers, so there is no single correction the projection is uniformly missing.

Test 3 — learnability (decisive). A patch predictor $R(\hat{W} \text{ patch} + \text{type/layer/position}) \rightarrow \Delta \text{ patch}$ is trained on 18 layers and scored on 6 stratified held-out layers as error reduction over predicting zero; the control retrains identically on Δ shuffled across layers, and a 1000-sample bootstrap brackets the real–control gap. Neither a linear nor an MLP-512 predictor beats predicting zero (reductions $-0.0004/-0.0003$), and real and control are indistinguishable (gap $CI_{95} \approx [-0.0000, -0.0000]$, $p \geq 0.99$; Table 5). There is no patch-learnable signal.

Test 4 — transfer. On the held-out pair 410M $\rightarrow$ 160M the shared operators fit at relative error 0.56–0.74 (better than the primary pair — a smaller width gap). A predictor trained on all primary-pair layers and evaluated on the held-out pair reduces error by $-0.0004/-0.0003$, again indistinguishable from its shuffle control ($p = 1.00$; Table 5). Nothing transfers, consistent with Test 3.

Reconciling the faint spectral signal. Tests 1 and 3 are not in tension. The 3–6% spectral deficit is a generic statistical trace — mild row/column-norm heterogeneity, for instance — not a correspondence between $\hat{W}$ and Δ that any predictor can exploit. A behavioral cross-check agrees: an assembled 410M whose six matrix types are operator-projected (with real biases, LayerNorms, and embeddings) scores 35,212 perplexity, and the learned correction moves it only $\sim 2\%$ (to 34,373); both remain non-functional. Notably the operator matrices alone beat random init (35k vs. 66k) once the surrounding tensors are real — unlike §5’s fully projected model — which again locates the destruction in the dense projection, not the residual.¹

¹The correction’s gains on the 18 training layers are partly memorized; only the held-out layers carry genuine predictions, and there the reduction is zero.

Table 4: Test 1 (spectral). Effective rank of the residual Δ vs. a per-type shuffled control and a shape/scale-matched Gaussian control. Δ sits 3–6% below both in every type — a faint, non-isotropic concentration, not a learnable direction.

Weight	Δ erank	Shuffled	Gaussian
Q	771.3	824.1	824.1
K	784.0	824.0	824.1
V	801.2	824.2	824.1
O	792.2	824.1	824.1
MLP_UP	947.2	989.4	989.4
MLP_DOWN	934.8	989.4	989.4

Table 5: Tests 3–4 (learnability and transfer). Patch-predictor error reduction over predicting zero, vs. a label-shuffled control; 1000-sample bootstrap CI on the real–control gap. No predictor beats zero, and none separates from its control.

Setting	Predictor	Reduction	Shuffle	Gap CI ₉₅	p
Held-out layers (1.4B→410M)	linear	−0.0004	−0.0004	[−0.0000, −0.0000]	0.99
	MLP-512	−0.0003	−0.0003	[−0.0000, −0.0000]	1.00
Transfer (410M→160M)	linear	−0.0004	−0.0004	—	1.00
	MLP-512	−0.0003	−0.0002	—	1.00

CIs are reported for the in-pair held-out test only; the transfer test reports p (M4). Reductions are negative because no predictor improves on predicting zero.

Handoff to §7. Within a model family the sizes share representation geometry but not parameter solutions, and what the best linear size-map misses behaves as noise: no cross-layer direction, no patch-learnable mapping, no cross-pair transfer, only a faint generic spectral fingerprint. A learned zero-shot correction is therefore not available at this granularity. Conversion value must instead come from initialization plus brief fine-tuning — the matched-budget recovery race of §7.

7 Matched-budget conversion: two independent levers

The characterization so far is diagnostic: representations align (§4), dense projection is functionally destructive (§5), and its post-operator residuals are noise (§6). The constructive claim follows here. If conversion value lives in initialization, the decisive test is a matched-budget recovery race: give every candidate the same target-size shape, the same data, schedule, and token budget, and let continued pre-training reveal which starting weights convert fastest. We find that the init decomposes into two independent levers — least-squares compensation, which fixes what the init computes (best zero-shot), and variance-preserving rescale, which fixes the scale the optimizer sees (best endpoints) — and that stacking both dominates the strongest single-lever variant on both axes and in every seed.

Protocol. All arms share the primary pair’s target shape (410M), identical data order, optimizer, cosine schedule, and token budget; only the starting weights differ (full protocol in §3). Crucially, every init is obtainable without the trained target: selection scores, compensation moments, and rescale factors are all read off the donor 1.4B and the frozen corpus, so no arm leaks target weights. Zero-shot columns are quick-eval perplexity (50k WikiText tokens) at $t=0$; final columns are full strided WikiText-103 perplexity at the 30M-token budget. Unless a $\pm$ appears, a cell is a single seed (footnoted).

7.1 The ablation ladder

Table 6 walks a seven-arm ladder from an untrained model to the full method. Read top to bottom it is monotone in the endpoint — each rung improves the final perplexity — but the

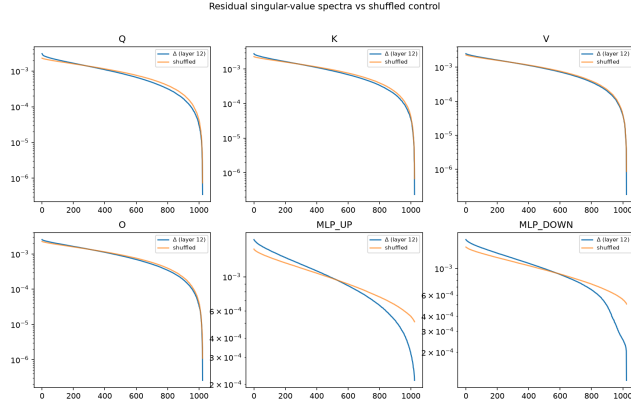


Figure 2: Test 1 (spectral). Singular-value spectrum of the residual Δ against shuffled and Gaussian controls. Δ concentrates 3–6% more than either control in every weight type — a faint, generic non-isotropy that carries no layer-specific, predictable signal (Table 5).

Table 6: Seven-arm ablation ladder on the primary pair (1.4B→410M), 30M tokens. Zero-shot is quick-eval perplexity at $t=0$; final is full WikiText-103 perplexity at budget. subclone_iso, hybrid, subclone_rs, and hybrid_rs all build on the same structured selection (rows marked “+”). Best zero-shot is hybrid (compensation); best endpoint is hybrid_rs (both levers). All cells single-seed; the two top arms are re-run across seeds in Table 7.

Init	Construction	Zero-shot ppl	Final ppl (30M)
random	from scratch	~66,000	1,519.0
projection	dense linear projection	12,323	1,054.5
subclone	structured selection	11,509	355.8
subclone_iso	+ rescale (read-out paths)	26,669	208.2
hybrid	+ compensation	9,664	114.1
subclone_rs	+ rescale (all cut paths)	61,912	86.4
hybrid_rs	+ compensation + rescale	18,459	83.0

zero-shot column is emphatically not monotone, and that dissonance is the section’s main result.

Both transfer inits beat from scratch. Even bare structured selection (subclone, 355.8) recovers $4.3\times$ faster than random (1,519.0) at matched compute, and dense projection (1,054.5) — despite its scrambled wiring — still beats from scratch $1.4\times$. Selection preserves valid wiring (rotary pairing, per-head attention, elementwise GELU, per-feature LayerNorm), and that structure pays off not merely at $t=0$ but throughout training: subclone separates from projection almost immediately and the gap widens monotonically (Fig. 3). The full ladder closes at $1,519.0/83.0 = 18\times$ better than from scratch.

7.2 Two independent levers

Reading the ladder by construction rather than by rank exposes two orthogonal axes over the selection base.

Compensation is the function lever. The hybrid re-fits the two purely-linear, LayerNorm-free read-out paths — the attention output projection O and the MLP down-projection — by closed-form ridge least squares (§7.4), so the surviving units absorb the correlated contribution of the units selection dropped. This strictly improves both axes: zero-shot falls $11,509 \rightarrow 9,664$ (the best zero-shot in the table) and the endpoint falls $355.8 \rightarrow 114.1$ ($3.1\times$ over subclone). Compensation changes what the initial network computes, and a better function is better everywhere.

Table 7: Primary-pair final WikiText-103 perplexity across three data-draw seeds (30M tokens; identical budget/schedule per seed). `hybrid_rs` wins every paired comparison and its worst seed beats `subclone_rs`’s best seed.

Init	seed 0	seed 1	seed 2	mean $\pm$ std
<code>subclone_rs</code>	86.4	93.7	88.9	89.7 ± 3.7
<code>hybrid_rs</code>	83.0	86.0	82.8	83.9 ± 1.8

Rescale is the training-dynamics lever. The reference recipe’s variance-preserving factor $\sqrt{d_{\text{in}}/d'_{\text{in}}}$ (Samragh et al., 2023) does the opposite. Applied to the LayerNorm-free read-out paths only (`subclone_iso`) it worsens zero-shot $11,509 \rightarrow 26,669$ yet improves the endpoint $355.8 \rightarrow 208.2$; applied to every input-cut path (`subclone_rs`) it collapses zero-shot to 61,912 — $5.4\times$ worse than bare selection — while driving the endpoint down to 86.4, past even the compensated hybrid. Rescale does not fix the function; a downstream LayerNorm renormalizes each rescaled read path, so the scalar cancels for the forward pass and merely perturbs the delicate zero-shot alignment. What it does fix is the variance the optimizer starts from, and along the rescale axis the trade is monotone: more rescale, worse start, better endpoint.

The levers stack. Because the two levers act on disjoint quantities — compensation on the function of the two LN-free read-out paths, rescale on the scale of the LN-fronted read-in paths (Q, K, V , the MLP up-projection, and the output embedding) that compensation deliberately leaves alone — they compose. `hybrid_rs` applies compensation where it is the ridge-optimal linear map and the reference-recipe scale where compensation cannot reach, and dominates the strongest single-lever arm (`subclone_rs`) on both axes: zero-shot $18,459 < 61,912$ (compensation offsets the rescale collapse) and endpoint $83.0 < 86.4$.

7.3 Seeded verdict

The two rescale leaders are close on the primary pair’s single seed (86.4 vs. 83.0), so we re-run both across three data-draw seeds (Table 7). The separation is clean: `hybrid_rs` wins every paired within-seed comparison (3/3), and its worst seed (86.0) still beats `subclone_rs`’s best seed (86.4) — the two seed distributions do not overlap.

An honest correction. The rescale lever is a correction to our own earlier reading. When the hybrid first won at 114.1 we argued that its least-squares compensation subsumed the reference recipe’s scalar rescale, on the grounds that a ridge-optimal linear map is strictly better than a scalar on the paths it re-fits. The upset — `subclone_rs` at 86.4, beating the hybrid with no compensation at all — showed that argument is only half right. Compensation does dominate rescale on the two read-out paths it touches; but the LN-fronted read-in paths, which compensation leaves untouched because the LayerNorm makes their reconstruction target ill-defined, still benefit from the reference recipe’s variance correction as a training-dynamics prior. Blanket rescale collapses zero-shot (it mis-scales reads a LayerNorm would have renormalized), yet it wins on dynamics — so the reference recipe is vindicated exactly where our first account had written it off, and the principled method is the stack, not either lever alone.

7.4 Compensated Selection

Algorithm 1 states the full construction. Selection is the architecture-aware step: it keeps whole coordinates so the symmetry structure of each subspace survives. Residual features are chosen once, globally, by activation variance (a shared stream admits one basis). Attention keeps frequency-matched rotary pairs — with rotary half-width $16 \rightarrow 8$ the small frequency ladder is exactly every other large frequency, so we keep pairs at stride 2 and preserve the (cos, sin) pairing (Su et al., 2024) — and tops up the non-rotary dims by $\|q\| + \|k\|$ row norm; the value/output path keeps the top head dims by $\|V_{\text{row}}\| \cdot \|O_{\text{col}}\|$; MLP units by $\|\text{up}_{\text{row}}\| \cdot \|\text{down}_{\text{col}}\|$. When the head count shrinks (the held-out pair) selection is promoted

Algorithm 1 Compensated Selection (hybrid_rs)

Require: donor weights W_L ; frozen-corpus activation variances and second moments Σ ;
 target shape (residual d' , head dim, MLP width, block map)
 Ensure: installable small-model weights (no target-model access)

- 1: $\mathcal{R} \leftarrow$ top- d' residual features by mean-over-layer activation variance {one global basis}
- 2: for each kept block b do
- 3: $\mathcal{K}_{qk} \leftarrow$ frequency-matched rotary pairs (stride 2) $\cup$ top non-rotary dims by $\|q\| + \|k\|$
- 4: $\mathcal{K}_{vo} \leftarrow$ top head dims by $\|V_{\text{row}}\| \cdot \|O_{\text{col}}\|$ {whole heads if head count shrinks}
- 5: $\mathcal{K}_{mlp} \leftarrow$ top units by $\|\text{up}_{\text{row}}\| \cdot \|\text{down}_{\text{col}}\|$
- 6: slice W_L block b to $(\mathcal{R}, \mathcal{K}_{qk}, \mathcal{K}_{vo}, \mathcal{K}_{mlp})$; LayerNorms and biases ride along
- 7: $O' \leftarrow \text{LScomp}(O, \Sigma_{\text{attn-in}}, \mathcal{K}_{vo})$; MLP_down' $\leftarrow \text{LScomp}(\text{MLP_down}, \Sigma_{\text{mlp-in}}, \mathcal{K}_{mlp})$ {Eq. 3, function lever}
- 8: if rescale then
- 9: multiply LN-fronted read-in paths $Q, K, V, \text{MLP_up}$ (and output embedding) by $\sqrt{d_{\text{in}}/d'_{\text{in}}}$ {dynamics lever}
- 10: end if
- 11: end for
- 12: return assembled small model

to whole heads scored by $\sum_d \|V_{\text{row},d}\| \|O_{\text{col},d}\|$, one keep-set shared by Q, K, V, O ; when depth shrinks, evenly strided blocks are kept and each kept block's norms and biases ride along.

Compensation is the closed-form optimum for a pruned linear map. For a map M reading a signal with measured second moment Σ , the ridge least-squares substitute on the kept columns $\mathcal{K}$ is

$$M' = \arg \min_{M'} \mathbb{E} \|Mx - M'x_{\mathcal{K}}\|^2 = M \Sigma_{:, \mathcal{K}} (\Sigma_{\mathcal{K}, \mathcal{K}} + \lambda I)^{-1}, \quad (3)$$

with plain subcloning as the special case $M' = M_{:, \mathcal{K}}$. We apply it at the two spots where the reconstruction target is well-defined — O and the MLP down-projection, both purely linear with no normalization between the cut and the read — and leave the LN-fronted read-in paths to the optional rescale lever. Least-squares reconstruction after pruning is classical (Hassibi & Stork, 1993; Ma et al., 2023); the contribution here is the setting (family size conversion, not same-model compression), the rotary-frequency-matched selection, and the two-lever decomposition around it.

7.5 Hardening

The headline is one metric at one budget on one pair. We stress it on all three.

Every metric, same ordering. Table 8 adds out-of-domain perplexity (C4), perplexity at $2\times$ the training context (2048), and four zero-shot tasks for the four original inits and the real-410M anchor. The ordering hybrid < subclone < projection < random holds on every metric. Long context is the cleanest signal: perplexity at 2048 tracks each model's 1024 number, so the rotary-frequency-matched selection extrapolates past its training length without penalty. LAMBADA (floor ≈ 0) is the sharpest ability gap — hybrid $14\times$ subclone — while the multiple-choice tasks sit near their guess floors for all four 30M checkpoints: 30M tokens buys language modeling, not task reasoning, and we claim only the former. Anchor scores match published Pythia numbers.

The lead persists at longer budget. Re-running the two leaders at 100M tokens, hybrid finishes at 62.3 and subclone at 103.1; hybrid crosses subclone's 100M endpoint at $\sim 38\text{M}$ tokens, a $\sim 2.6\times$ token-efficiency gain.²

²100M cells are single-seed. Cross-horizon comparisons are schedule-confounded: the cosine schedule stretches with the budget, so we compare finals only within a horizon (e.g. we do not read hybrid-30M's 114.1 against subclone-100M's 103.1 as a controlled result).

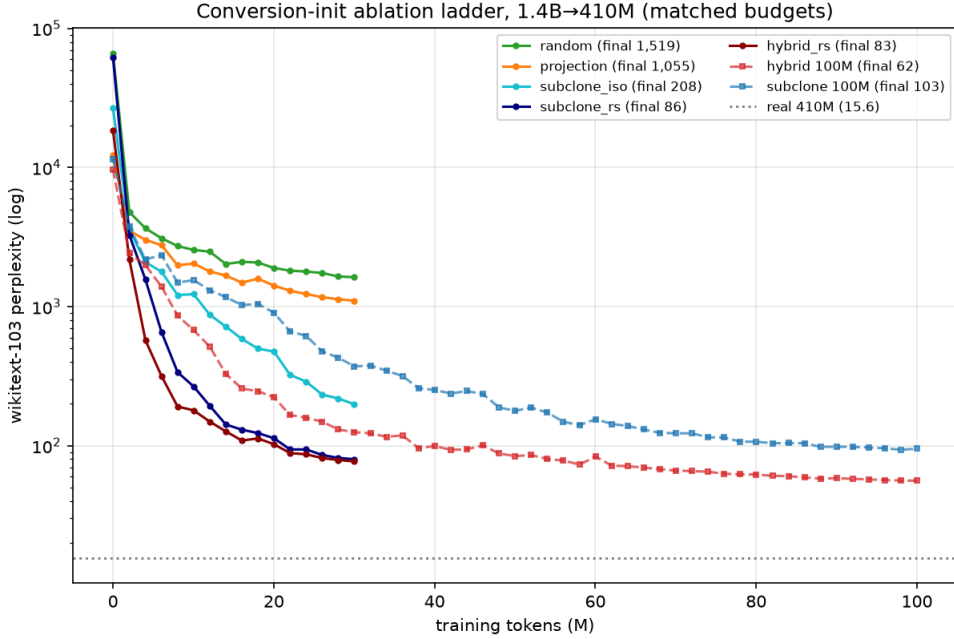


Figure 3: The recovery ladder on the primary pair (1.4B→410M). Full WikiText-103 perplexity (log scale) versus tokens for the seven arms of Table 6, all sharing data order, schedule, and the 30M-token budget. The dashed line marks the real Pythia-410M reference (≈ 15.6). Two families are visible: transfer inits start orders of magnitude below random and separate immediately; compensation (hybrid) lowers the whole curve, while the rescale arms (subclone_iso, subclone_rs, hybrid_rs) start higher yet overtake by end of budget. All arms remain far from the reference: 30M tokens is $\sim 0.01\%$ of Pythia’s pre-training, so the race measures relative recovery speed, not end quality.

Table 8: Extended evaluation of the 30M primary-pair checkpoints and the real-410M reference. Guess floors: arc_easy 0.25, hellaswag 0.25, piqa 0.50, lambada ≈ 0 (open-vocabulary). Best converted arm in bold; the ordering holds on all six metrics. Single-seed checkpoints. This table predates the rescale-lever ablation and reports the compensation contrast (hybrid vs. subclone); the rescale gains of Tables 6–7 stack on top of it.

Model	C4 ppl	wk@2048	lambada	arc_easy	hellaswag	piqa
hybrid	90.7	109.8	0.076	0.313	0.266	0.555
subclone	162.7	346.3	0.005	0.308	0.263	0.548
projection	380.5	1,210	0.000	0.272	0.258	0.539
random	518.8	1,788	0.000	0.264	0.256	0.532
real 410M	21.0	14.5	0.516	0.519	0.337	0.667

A held-out pair, and the method’s boundary. The 410M→160M pair exercises axes the primary pair does not: depth halves (24 → 12 blocks), the head count drops (16 → 12, whole-head selection), and width cuts are mild ($\sim 25\%$). Across three seeds (Table 9), transfer beats from scratch $\sim 13\times$ on every seed — the core claim generalizes with error bars to a regime the method was never tuned on. But here hybrid and subclone are a statistical tie (overlapping $\pm 1\sigma$; hybrid nominally ahead), unlike the $3.1\times$ compensation gap on the width-dominated primary pair. The boundary is mechanistic: compensation repairs width cuts because surviving units can absorb dropped ones, but deleted blocks have no survivors to re-tune, and this pair’s damage is depth-dominated. Compensation’s edge therefore scales with the width reduction — and costs essentially nothing where it is neutral. The zero-shot starting lines echo this: hybrid starts at $\approx 30k$ here versus 9.7k on

Table 9: Held-out pair (410M→160M), final WikiText-103 perplexity across three data-draw seeds (30M tokens). Transfer beats from scratch $\sim 13\times$ on every seed; hybrid and subclone are a statistical tie on this depth-dominated pair (overlapping $\pm 1\sigma$).

Init	seed 0	seed 1	seed 2	mean $\pm$ std
hybrid	117.5	113.6	108.5	113.2 ± 4.5
subclone	115.0	121.6	118.9	118.5 ± 3.3
random	1,464.4	1,571.5	1,479.6	$1,505.2 \pm 57.9$

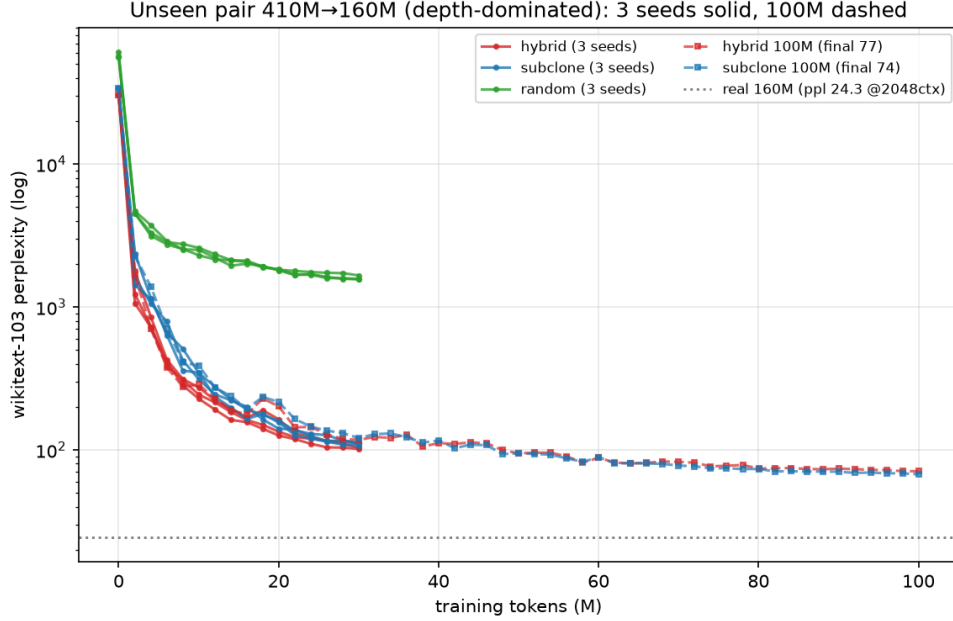


Figure 4: Held-out pair (410M→160M), three seeds. Full WikiText-103 perplexity (log scale) versus tokens; bands span the three data-draw seeds and the dashed line marks the real Pythia-160M reference. Transfer inits (hybrid, subclone) sit an order of magnitude below random throughout and overlap each other — the depth-dominated regime where compensation’s width-repair edge is neutral (Table 9), yet the $\sim 13\times$ margin over from scratch is preserved on every seed.

the width pair. The tie is budget-robust: at 100M tokens (seed 0) hybrid 77.1 vs. subclone 74.4, a 3.6% gap inside the seed noise with the nominal leader flipped — not an artifact of short training. It also holds on every other metric (C4 91.1 vs. 94.2, ctx-2048 111.8 vs. 110.2, LAMBADA 11.0% vs. 10.7%, others within noise), while random sits at C4 514 and LAMBADA 0; the real-160M anchor (LAMBADA 35.4%) matches published Pythia numbers.³

7.6 Scale

The scale pair (6.9B→1.4B) reduces all three axes at once and tests whether the levers hold an order of magnitude up. Runs are in flight; Table 10 is a placeholder.

Compute accounting. Initialization construction is negligible against any training budget and is included in the accounting. Selection is norms plus top- k (seconds); the compensation moments pass over 1,000 frozen-corpus sequences is ~ 75 s; the closed-form solves of Eq. 3

³The 100M and seed-0 metric-robustness cells are single-seed. Width-vs-depth budget allocation and reduction-aware conversion (block distillation) are future work (§8).

Table 10: Scale pair (6.9B→1.4B), matched budget. Runs in flight.

Init	Zero-shot ppl	Final ppl (30M)
random	—	—
subclone	—	—
hybrid_rs	—	—

are seconds; the rescale lever is a scalar multiply (free). Every arm then trains on the same 30M tokens under the same schedule (~ 55 min each on a single GB10 at ~ 9.4 k tok/s), so all standings in Tables 6–7 compare equal-budget endpoints whose only difference is the starting weights.

8 Limitations and future work

One family, one wiring. All experiments are within the Pythia suite (GPT-NeoX blocks: LayerNorm, elementwise GELU, full multi-head attention with rotary embeddings). This is deliberate — identical data and tokenizer isolate size — but the two-lever decomposition leans on exactly those wiring details: which read paths a normalization fronts determines where compensation is well-posed and where rescale acts (§7.4). Llama- and Qwen-class models change the load-bearing parts — RMSNorm in place of LayerNorm, gated SwiGLU MLPs whose hidden units couple two input matrices, grouped-query attention that ties key/value heads across query heads — so porting the method requires a per-architecture safe-wiring analysis (which units may be selected together; which paths a norm protects), not a mechanical re-run. The characterization tools themselves (alignment maps, shuffle controls, the matched-budget race) are architecture-agnostic.

Budgets far from convergence. Our recovery budgets (30M tokens primary, 100M persistence) are $\sim 0.01\%$ of Pythia’s 300B-token pretraining, and every converted model remains far from the real target’s quality (best endpoint 83.0 vs. the real 410M’s 15.6). All claims are therefore about relative recovery speed at matched budget, not end quality; whether the orderings persist to full recovery is untested.

Seed counts. The decisive comparisons carry three data-draw seeds each (the seeded verdict of Table 7; the held-out pair of Table 9); the remaining ladder rungs, the 100M persistence checks, and the scale pair (§7.6) are single-seed. We report $\pm$ only where seeds exist and footnote every single-seed cell.

Read-in compensation is unexplored. Compensation is applied only where the reconstruction target is well-defined: the two LayerNorm-free read-out paths. On the LN-fronted read-in paths, the same renormalization that makes blanket rescale collapse zero-shot (§7.2) makes the least-squares target ill-defined — the norm re-standardizes whatever the cut changes, so there is no fixed signal to reconstruct. Whether a normalization-aware compensation exists for these paths is open, and it is the most direct route to further improving the zero-shot axis.

Depth is the uncompensable axis. The held-out pair locates the method’s boundary (§7.5): compensation repairs width cuts because surviving units can absorb dropped ones, but a deleted block leaves no survivors to re-tune. Two directions follow naturally, and we have attempted neither: reduction-aware conversion that recovers deleted blocks’ function into surviving neighbors (a distillation-flavored step), and choosing the width-versus-depth reduction mix for a given parameter budget — our measured boundary suggests preferring width where the budget allows. We would welcome follow-up on both.

Reproducibility statement

References

- Samuel K. Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git Re-Basin: Merging models modulo permutation symmetries. In International Conference on Learning Representations (ICLR), 2023.
- Yamini Bansal, Preetum Nakkiran, and Boaz Barak. Revisiting model stitching to compare neural representations. In Advances in Neural Information Processing Systems 34 (NeurIPS), 2021.
- Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. In Proceedings of the 40th International Conference on Machine Learning (ICML), pp. 2397–2430. PMLR, 2023.
- Cheng Chen, Yichun Yin, Lifeng Shang, Xin Jiang, Yujia Qin, Fengyu Wang, Zhi Wang, Xiao Chen, Zhiyuan Liu, and Qun Liu. bert2BERT: Towards reusable pretrained language models. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL), 2022.
- Tianqi Chen, Ian Goodfellow, and Jonathon Shlens. Net2Net: Accelerating learning via knowledge transfer. In International Conference on Learning Representations (ICLR), 2016.
- Babak Hassibi and David G. Stork. Second order derivatives for network pruning: Optimal brain surgeon. In Advances in Neural Information Processing Systems 5 (NIPS), pp. 164–171, 1993.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531, 2015.
- Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In Proceedings of the 36th International Conference on Machine Learning (ICML), pp. 3519–3529. PMLR, 2019.
- Karel Lenc and Andrea Vedaldi. Understanding image representations by measuring their equivariance and equivalence. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 991–999, 2015.
- Xinyin Ma, Gongfan Fang, and Xinchao Wang. LLM-Pruner: On the structural pruning of large language models. In Advances in Neural Information Processing Systems 36 (NeurIPS), 2023.
- Maithra Raghu, Justin Gilmer, Jason Yosinski, and Jascha Sohl-Dickstein. SVCCA: Singular vector canonical correlation analysis for deep learning dynamics and interpretability. In Advances in Neural Information Processing Systems 30 (NeurIPS), 2017.
- Mohammad Samragh, Mehrdad Farajtabar, Sachin Mehta, Raviteja Vemulapalli, Fartash Faghri, Devang Naik, Oncel Tuzel, and Mohammad Rastegari. Weight sub-cloning: direct initialization of transformers using larger pretrained ones. arXiv preprint arXiv:2312.09299, 2023.
- Sheng Shen, Pete Walsh, Kurt Keutzer, Jesse Dodge, Matthew Peters, and Iz Beltagy. Staged training for transformer language models. In Proceedings of the 39th International Conference on Machine Learning (ICML), pp. 19893–19908. PMLR, 2022.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. Neurocomputing, 568:127063, 2024.

Peihao Wang, Rameswar Panda, Lucas Torroba Hennigen, Philip Greengard, Leonid Karlinsky, Rogerio Feris, David D. Cox, Zhangyang Wang, and Yoon Kim. Learning to grow pretrained models for efficient transformer training. In International Conference on Learning Representations (ICLR), 2023.

Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. Sheared LLaMA: Accelerating language model pre-training via structured pruning. In International Conference on Learning Representations (ICLR), 2024.